EE2016 Microprocessors Theory and Lab, Aug - Nov 2025

Tutorial 4 (Classes on 1st, 3rd, 8th Sep - 1st batch (29th Aug, 2nd, 9th Sep - 2nd batch))

Dr. R. Manivasakan, EE Dept, IIT Madras.

Portions: Keyboard debouncing problem, polling implementation in KB debouncing problem, interrupt implementation in KB debouncing problem, and polling (KIN bit) and interrupt (KIRQ) implementation in muP - KB PIC problem, CU design, microprogramming and hardwired control unit, hardwired CU design examples: #1 Booths multiplier, #2 GCD processor, hardware design procedure - abstraction of CU as an FSM - Moore's (rather than Mealy's) machine, consolidation of states involved, table of current state versus next states and value of output signals (Boolean variables), translation of above states into excitation table (TT) and subsequent digital design.

1 Fill in the blanks

1. The two main types of addressing a peripheral (given a set of peripherals, picking a desired one) are Memory Mapped Input/ Output(MMIO) and port mapped I/O (PMIO)
2. In memory mapped I/O the address space is shared (shared by / separate for) memory and PIC registers of peripherals.
3. Example of port mapped I/O based processors is intel x86 and intel x86-64 microprocessor.
4. Three important registers in PIC of a controller are data, control and status registers.
5. In a byte addressible memory chip of size 256 bytes, the word length is 1 byte, the address bus width is 8 bits, data bus width is 8 bits and the control pins are CS, RD, WR and Clk.
6. The status signal (in the context of PIC) is unidirectional (unidirectional / bidirectional) and if it is unidirectional, then its direction is from PIC register to CU of CPU.
7. For a keyboard the system data bus connecting to CPU is unidirectional (unidirectional / bidirectional) and if it is unidirectional, then its direction is from keyboard to CPU.
8. In polling method of handshake, the proactive component is CPU (CPU / Peripheral).
9. Advantage of interrupt over polling is (are) (a) CPU time is saved as the peripheral is proactive in intimating the CPU that an event has occured, which CPU has to ATTEND urgently and polling by CPU is absent (b) CPU energy is saved, since its energy / power could be spent constructively in executing other programs, which it was already executing, which otherwise be waiting for the peripheral to complete (c) CPU throughput, the number of instructions executed per sec is increased, due to absence of polling time.
10. Prioritizing the service by microprocessor, for such service requests from various I/O devices, in polling, is impossible (possible/ impossible), while in interrupts it is possible (possible/ impossible).
11. The criteria for concluding that "a key has been pressed", (in the polling method of solution to the debouncing problem) is 13 number of samples of '0' (no contact) followed by 7 number of samples of '1' (contact). (13 0s followed by 7 1s - meaning 13 ms consecutively no contact followed by 7 ms there should be an uninterrupted contact)

12. PC value abruptly jumps during instructions: JMP, subroutine & interrupt (select from following list: LDI, JMP, subroutine, ADD, interrupt).

13. The role of stack pointer in interrupt is (are) (a) store the contents of the PC before the PC value jumps abruptly (either in the case of subroutine or interrupt) apart from values of context registers and status registers (b) retrieve the contents to the PC, context registers & status registers after subroutine or ISR is over.

14. Stack is a FILO or LIFO (FIFO or LIFO or FILO or LILO) system.

15. The number of POP and PUSH operations in a stack, for a specific interrupt is equal (equal / necessarily unequal).

16. The stack pointer, points to variable bottom most location (variable bottom most location / variable midlocation / fixed top most location) of active area of stack, which uses the convention of PUSHing from top. [The stack pointer points to the variable end of stack memory (implying that the intervening memory locations - from fixed end - are to be PUSHed out, eventually)]

17. The purpose of KIN bit in the KBD_STATUS register of keyboard PIC is to tell the CPU, whether the data in the data register of KBD_DATA is fresh (and is not stale/ obselete & CPU must read it).

18. The KBD_STATUS register can only be written into, (read / written into) by keyboard peripheral.

19. The KIN bit in KBD_STATUS register is used for polling and interrupt schemes (polling / interrupt / both polling & interrupt schemes) method of handshake.

20. Order the rates in ascending order: (a) rate of keys pressed (b) rate of CPU cycles (c) rate at which the display device could display characters (a), (c) and (b)

21. The meaning of the KIN bit in KBD_STATUS being '1' is that the data (ASCII code) in KBD_DATA is valid (valid / invalid - valid means the CPU would READ, invalid means CPU has already READ and should NOT READ again).

22. The main difference between a subroutine and an interrupt-service routine is external interrupt service request arrivals are random (Hint: in terms of their arrival time?) during execution of a program.

23. The hardwired control unit (instruction decoder part) consists of instruction mapper, state counter / sequencer and combinational & sequential digital logic circuit implementing the control matrix.

24. Booths algorithm is to compute multiplication (GCD, multiplication, Ex-OR) of two numbers.

25. Final derived circuit for CU for Booths algorithm consists of a combination of combinational and sequential (purely combinational, purely sequential, a combination of combinational and sequential) digital circuit. [The term sequential circuit includes logic gates].

2 Answer the following

1. Hardwired control is used in contemporary instruction-set microprocessors. Consider the design of a hardwired control unit based on the classical method for a gcd processor that computes the gcd(X, Y) of two positive integers X and Y . The gcd procedure assigns the input data X and Y to a pair of registers XR and YR. While XR $>$ 0, the procedure (i) swaps XR and Y if XR $\leq$ YR and (ii) subtracts YR from XR and assigns the result to XR. The GCD procedure ends whenever XR$<=$0, yielding the GCD (as the final value of YR).

   (a) Show the hardware to implement the GCD algorithm.

(b) Show the datapath hardware needed to implement the above GCD procedure.

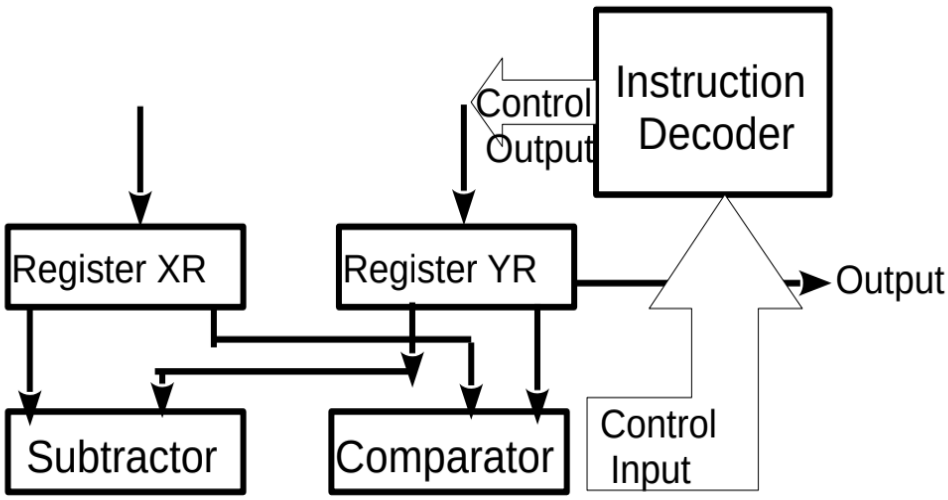
(c) Show the CU with input and output control signals for the above datapath hardware.

(d) Derive the CU for the above GCD processor by the standard procedure (taught in the class, steps given below) after assuming Moore type FSM.

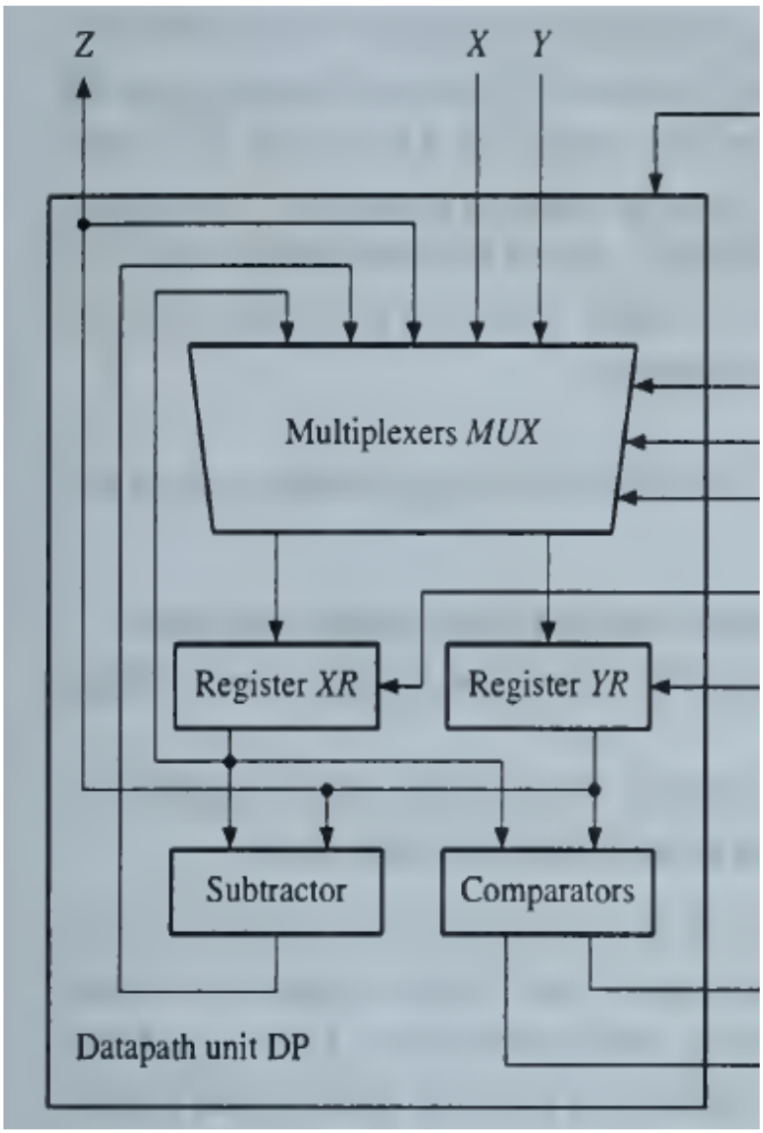
i. Identify the right FSM modelling the algorithm (or pseudocode)
ii. Identify the states and write the corresponding state table
iii. Identify the excitation table (Truth table)
iv. (Optional) Using Karnaugh's map derive the expression for each output
v. (Optional) Using the result in 1(d)(iv) draw the corresponding circuit

Solutions:

(a) Hardware for GCD (swap to be included)



(b) Datapath Hardware



The datapath hardware is the collection of functional components within a microprocessor that performs operations on data right from the operands, initial values & relevant parameters needed for the computational task and moves that manipulated version of the data between different parts of the CPU till the end result form is achieved, thus accomplishing the task. The GCD datapath hardware is different from just the "GCD hardware" as the later hides the information of exactly where and how the CU control signals intervene the data, as it flows from input to output.

(c) The CU is supposed to send the control signals at the right time, and at the right (digital) datapath circuit point, and these signals form the output of CU. Here, the following are the output of CU: (i) Select XY (ii) Swap (iii) Subtract (iv) Load XR (v) Load YR and (vi) Done. Each output can be justified. Theoretically, there should be some factors driving the CU and most of the time, these factors evolve as the algorithm runs and these form the input. Here the input is the result of comparisons (i) (XR>0) (ii) (XR>=YR). Now, the CU is pictorially represented as a (black) box with the above inputs and outputs.

(d) CU for the GCD is derived thus:

i. The FSM consists of states and the branches connecting them. This FSM is derived by modelling the algorithm.

ii. First, the following states are chosen and consolidated (after considering carefully the structure of the algorithm)

A. S_0 Begin

B. S_1 Swap

C. S_2 Subtract

D. S_3 End

State table is given by:

State	**Input (XR>0) (XR>=YR) digital signals**								
	0*	**10**	**11**	**Subtract**	**Swap**	**Select XY**	**Load XR**	**Load YR**	**Done**
S0 (Begin)	S3	S1	S2	0	0	1	1	1	0
S1 (Swap)	S2 (NA)	S2	S2 (NA)	0	1	0	1	1	0
S2 (Subtract)	S3 (NA)	S1	S2	1	0	0	1	0	0
S3 (End)	S3	S3	S3	0	0	0	0	0	1

This is different from what has been given in the text by Hayes. Done output is make Z available only at the 'End' state (Otherwise it is invalid).

iii. Truth Table (or Excitation Table)

Inputs		Present State		Next State		Current control signals					
(XR>0)	(XR>=YR)	D1	D0	D1+	D0+	Subtract	Swap	Select XY	Load XR	Load YR	Done
0	d	0	0	1	1	0	0	1	1	1	0
0	d	0	1	1	0	0	1	0	1	1	0
0	d	1	0	1	1	1	0	0	1	0	0
0	d	1	1	1	1	0	0	0	0	0	1
1	0	0	0	0	1	0	0	1	1	1	0
1	0	0	1	1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	0	0	1	0	0
1	0	1	1	1	1	0	0	0	0	0	1
1	1	0	0	1	0	0	0	1	1	1	0
1	1	0	1	1	0	0	1	0	1	1	0
1	1	1	0	1	0	1	0	0	1	0	0
1	1	1	1	1	1	0	0	0	0	0	1

iv. The resulting Karnaugh's map is very involved: with 6 input Boolean variables for each output control signal. But, offline you can work it out (I would not ask this part as it is more, a part of digital systems course).

v. The resulting Boolean expression for each control output is expected to be also very involved. (Ditto as above).

2. Hardwired CU for Booths multiplier:

(a) Give the Booths multiplication algorithm [Refer Stallings / ppt file uploaded. A modified version of the algorithm is implemented in majority of the processors available in the market even today].

(b) Show the hardware to implement the Booths algorithm.

(c) Show the datapath hardware needed to implement the above Booths algorithm.

(d) Show the CU with input and output control signals for the above datapath hardware.

(e) Derive the CU for the above GCD processor by the standard procedure (taught in the class, steps given below) after assuming Moore type FSM.

i. Identify the right FSM modelling the algorithm (or pseudocode)

ii. Identify the states and write the corresponding state table

iii. Identify the excitation table (Truth table)

iv. (Optional) Using Karnaugh's map derive the expression for each output

v. (Optional) Using the result in 2(e)(iv) draw the corresponding circuit

Solutions:

(a) See Stallings or presentation pdf file whose link is given in ANNOUNCEMENTs in moodle. Slide no: 16

(b) Refer presentation pdf file, slide no: 16

(c) Refer presentation pdf file, slide no: 17

(d) Refer presentation pdf file, slide no: 17

(e) CU for the above GCD processor

i. The FSM contains 6 states (S_0 (Start) , S_1 (Subtract), S_2 (AShR) , S_3 (Add) , S_4 (Dec) ,S_5 (Done))and the branches are dictated by the transitions as in state table (given in e(ii)). S_2 is Arithmetic Shift Right.

ii. The states are as above (as in e(i)). The corresponding state table is given below.

State	Q_0Q_{-1} (10) (n> 0)	Q_0Q_{-1} (00, 11) n>0	Q_0Q_{-1} (01) n>0	Q_0Q_{-1} XX n=0	Add	Sub	AShR	Decn	LoadA	Select AMQ	Done
S0 (Start)	S1	S2	S3	S5	0	0	0	0	0	1	0
S1 (Sub)	S2	S2	S2	S5	0	1	0	0	1	0	0
S2 (AShR)	S4	S4	S4	S5	0	0	1	0	0	0	0
S3 (Add)	S2	S2	S2	S5	1	0	0	0	1	0	0
S4 (Dec)	S1	S2	S3	S5	0	0	0	1	0	0	0
S5 (Done)	S5	S5	S5	S5	0	0	0	0	0	0	1

iii. Excitation table is derived from the above state table

Input			Current State			Next State			Add	Sub	AShR	Decn	LoadA	Select AMQ	Done	Just (dummy Reference column
1	0	(n> 0)	0	0	0	0	0	1	0	0	0	0	0	1	0	S0 (Start)
1	0	(n> 0)	0	0	1	0	1	0	0	1	0	0	1	0	0	S1 (Sub)
1	0	(n> 0)	0	1	0	1	0	0	0	0	1	0	0	0	0	S2 (AShR)
1	0	(n> 0)	0	1	1	0	1	0	1	0	0	0	1	0	0	S3 (Add)
1	0	(n> 0)	1	0	0	0	0	1	0	0	0	1	0	0	0	S4 (Dec)
1	0	(n> 0)	1	0	1	1	0	1	0	0	0	0	0	0	1	S5 (Done)
0	0	(n> 0)	0	0	0	0	1	0	0	0	0	0	0	1	0	S0 (Start)
0	0	(n> 0)	0	0	1	0	1	0	0	1	0	0	1	0	0	S1 (Sub)
0	0	(n> 0)	0	1	0	1	0	0	0	0	1	0	0	0	0	S2 (AShR)
0	0	(n> 0)	0	1	1	0	1	0	1	0	0	0	1	0	0	S3 (Add)
0	0	(n> 0)	1	0	0	0	1	0	0	0	0	1	0	0	0	S4 (Dec)
0	0	(n> 0)	1	0	1	1	0	1	0	0	0	0	0	0	1	S5 (Done)
1	1	(n> 0)	0	0	0	0	1	0	0	0	0	0	0	1	0	S0 (Start)
1	1	(n> 0)	0	0	1	0	1	0	0	1	0	0	1	0	0	S1 (Sub)
1	1	(n> 0)	0	1	0	1	0	0	0	0	1	0	0	0	0	S2 (AShR)
1	1	(n> 0)	0	1	1	0	1	0	1	0	0	0	1	0	0	S3 (Add)
1	1	(n> 0)	1	0	0	0	1	0	0	0	0	1	0	0	0	S4 (Dec)
1	1	(n> 0)	1	0	1	1	0	1	0	0	0	0	0	0	1	S5 (Done)
0	1	(n> 0)	0	0	0	0	1	1	0	0	0	0	0	1	0	S0 (Start)
0	1	(n> 0)	0	0	1	0	1	0	0	1	0	0	1	0	0	S1 (Sub)
0	1	(n> 0)	0	1	0	1	0	0	0	0	1	0	0	0	0	S2 (AShR)
0	1	(n> 0)	0	1	1	0	1	0	1	0	0	0	1	0	0	S3 (Add)
0	1	(n> 0)	1	0	0	0	1	1	0	0	0	1	0	0	0	S4 (Dec)
0	1	(n> 0)	1	0	1	1	0	1	0	0	0	0	0	0	1	S5 (Done)
d	d	0	0	0	0	1	0	1	0	0	0	0	0	1	0	S0 (Start)
d	d	0	0	0	1	1	0	1	0	1	0	0	1	0	0	S1 (Sub)
d	d	0	0	1	0	1	0	1	0	0	1	0	0	0	0	S2 (AShR)
d	d	0	0	1	1	1	0	1	1	0	0	0	1	0	0	S3 (Add)
d	d	0	1	0	0	1	0	1	0	0	0	1	0	0	0	S4 (Dec)
d	d	0	1	0	1	1	0	1	0	0	0	0	0	0	1	S5 (Done)

iv. The resulting Karnaugh's map is very involved: with 9 input Boolean variables for every output control signal. But, offline you can work it out (I would not ask this part as it is more, a part of digital systems course).

v. The resulting Boolean expression for each control output is expected to be also very involved. Ditto as above.

3. (Optional) Repeat the Pr. 2 for Floating Point Unit. Meaning consider the two floating point numbers with fixed finite precision for matissa and exponent (& thus memory to hold those numbers are fixed). Now, repeat the Pr. 2 for multiplication of two floating point numbers.

4. **Polling versus Interrupt:** In this problem, we consider two designs of handshaking procedure between CPU and a peripheral device: polling and interrupt. The CPU was executing its own program, till 32nd instruction. 33rd instruction requires a job be assigned to a peripheral device [Good example of a peripheral is the printer] by the CPU. Method by which the peripheral informs the CPU that it is done, and what the CPU doing in the intervening period is the subject of this problem. Three schemes analysed in this problem are: (a) CPU waits perpetually, for the task to be completed by the peripheral (without *polling & interrupt*) (b) *polling:* CPU concurrently executes subsequent instructions, but polls periodically to see whether peripheral has completed its assigned job and (c) *interrupt* CPU concurrently executes subsequent instructions, but is interrupted by I/O (whenever it needs service or during service, whenever it is done). The figure below illustrates the comparison between the schemes as they all evolve in time

   Following are the three cases in which a processor - peripheral handshaking is designed.

   (a) *Case A (Without interrupt & polling):* In this case, after initiating the peripheral I/O operations in T_{REQ}, the processor waits for the peripheral's I/O operation to complete. During this waiting period, the microprocessor perpetually checks whether the above operation is complete or not. Each such checking takes a time interval of T_{chk}. If the I/O job, (which takes $T_{I/O}$ secs) is over, the time T_{RES} is taken by the peripheral's I/O module for writing the result of the peripheral operation and the CPU reading it to conclude the I/O operation.

   (b) *Case B (Polling):* In this case, the processor is allowed to do other jobs (after initiating the peripheral's I/O operations in T_{REQ}), but in this scheme, the processor polls, at regular intervals $T_{poll} = N_2 T_{INS} + T_{chk}$ to check whether the I/O job is over or not. The checking time interval is T_{chk} secs.

   (c) *Case C (Interrupt)* In this case also, the processor is allowed to do subsequent instructions (concurrently with the peripheral's operation, after initiating the memory I/O operations in T_{REQ}). The difference is, the CPU can continue to execute the program, say, instruction 34, 35 and so on, till the peripheral '*interrupts*' it, to say it is done with the I/O task assigned. [If the I/O job taking $T_{I/O}$ secs is over, the processor would be interrupted by the concerned peripheral]. The time T_{RES} is taken by the I/O module for writing the result of the peripheral operation and the CPU reading it to conclude the I/O operation.

   Given the following assumptions,

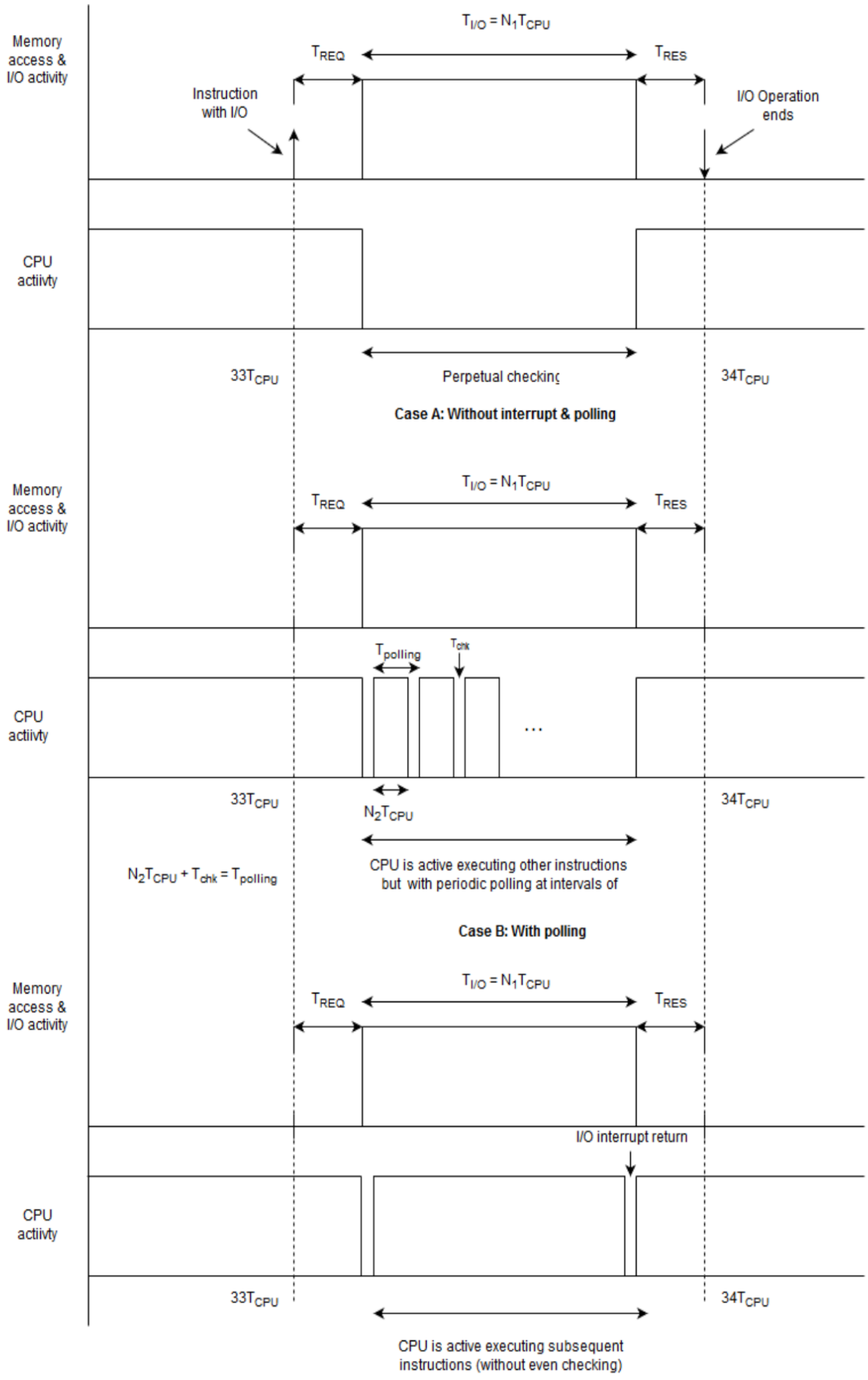
   (a) For normal instructions (without I/O) the time required for decode and execute instructions is denoted by $T_{INS} = 6T_{clk}$.

   (b) There are a total of 100 such instructions (each of which takes T_{INS}) out of which the 33rd instruction (& only one such) involves I/O operation.

   (c) Execution time for I/O operations is composed of three components (a) preparation for I/O operation which involves CPU time T_{REQ} (b) actual I/O (eg. print job) operation time being $T_{I/O} = N_1 T_{INS} \quad N_1 > 1$ (c) response by the I/O peripheral in setting the flag in PIC (whether the operation is success or failure) taking time T_{RES}.

   (d) The time for preparation for I/O operation T_{REQ} represents time for exchange of protocol (hand shaking) control information initially between CPU and the peripheral. For the peripheral being output devices (like memory module, image printing in a color printer, video card etc), T_{REQ}

means the time required to transfer data, for eg. time to transfer the digital image data to be printed through the peripheral, printer and the parameters of the print command. For the peripheral being input device (eg, camera, sensors, scanners, etc.,) T_{REQ} may mean initial control information from device, like the type of service requested, amount of memory space required, etc.


Memory access & I/O activity
$T_{I/O} = N_1 T_{CPU}$
T_{REQ}
T_{RES}
Instruction with I/O
I/O Operation ends
CPU actiivty
$33T_{CPU}$
Perpetual checking
$34T_{CPU}$
Case A: Without interrupt & polling
Memory access & I/O activity
$T_{I/O} = N_1 T_{CPU}$
T_{REQ}
T_{RES}
$T_{polling}$
T_{chk}
CPU actiivty
$33T_{CPU}$
$N_2 T_{CPU}$
$34T_{CPU}$
$N_2 T_{CPU} + T_{chk} = T_{polling}$
CPU is active executing other instructions but with periodic polling at intervals of
Case B: With polling
Memory access & I/O activity
$T_{I/O} = N_1 T_{CPU}$
T_{REQ}
T_{RES}
I/O interrupt return
CPU actiivty
$33T_{CPU}$
$34T_{CPU}$
CPU is active executing subsequent instructions (without even checking)
Case C: With interrupt

(e) The I/O operation time $T_{I/O}$, may mean for instance, the physical printing time. In many electromechanical peripherals, time of execution is substantially larger than that of clock interval. In the case of, memory transfer among memory chips, the memory transfer happens between the data register of the PIC and the CPU main memory. For input devices like camera, sensors, scanners, etc., it is the actual data transfer of enduser data from PIC to CPU. For scanner, physical scanning and data being available in its PIC.

(f) The time interval T_{RES} accounts for again the protocol (hand shaking) control overhead for concluding the I/O transaction. For a color image printer $T_{REQ} \gg T_{RES}$. In this problem, for cases A through C, take $T_{REQ} = 10T_{INS}$ and $T_{RES} = \frac{T_{INS}}{2}$.

(g) Primitive checking loop (degenerate form of polling) time is T_{chk} where $T_{chk} = \frac{T_{INS}}{3} = 2T_{clk}$.

(h) The result of the execution of instruction at 33, is used up in instruction $K > 34$ (and the processor would not be able to proceed further without executing instruction K).

Note: In figure above, read T_{CPU} as T_{INS}.

Compute

(a) Total clock cycles required in Cases A through C (for the whole program to complete). Use the notation $T_{prptl}, T_{poll}, T_{intrrpt}$ for the total clock cycles in each case

(b) Which of the schemes (A through C) above is most efficient, in terms of execution time (the lesser, the better) for the whole program to complete. Support your arguments with quantitative results. [Show the inequalities involving $T_{prptl}, T_{poll}, T_{intrrpt}$].

(c) Compute the value (or range of values) of N_1 and N_2 such that the CPU need not wait for results to execute statement number K.

Solutions:

(a) Cases A: Without interrupt and polling

Time taken for peripheral (I/O) operation,

$$\begin{aligned} T_{33} &= T_{REQ} + N_1 T_{INS} + T_{RES} \\ &= 10 * 6T_{clk} + 6N_1 T_{clk} + 3T_{clk} \\ &= (63 + 6N_1)\, T_{clk} \quad secs \end{aligned}$$

<Students miss out 6 in 6N1 [1.5 marks]>

Total time required for whole program

$$\begin{aligned} &= 99 * 6T_{clk} + T_{33} \\ &= 99 * 6T_{clk} + (63 + 6N_1)\, T_{clk} \\ T^{net} &= (594 + 63 + 6N_1) T_{clk} secs \end{aligned}$$

$$T^{net} = (657 + 6N_1)\, T_{clk} \quad secs$$

$$T^{net} = 6\,(109.5 + N_1)\, T_{clk} \quad secs$$

Total number of clock cycles (T_{clk})required

$$\begin{aligned} T^{net}_{prptl} &= (657 + 6N_1) \\ &= (657 + 6N_1) \quad clock\ periods \quad or \quad clock\ cycles \end{aligned}$$

$$T_{prptl}^{net} = (657 + 6N_1)\, T_{clk} \text{secs} \tag{1}$$

Case B: With Polling

Time taken for peripheral (I/O) operation, as with case A,

$$\begin{aligned} T_{33} &= T_{REQ} + N_1 T_{INS} + T_{RES} \\ &= 10 * 6T_{clk} + 6N_1 T_{clk} + 3T_{clk} \\ &= (63 + 6N_1)\, T_{clk} \quad secs \end{aligned}$$

Define

$$K_{poll} = \left\lfloor \frac{N_1 T_{INS}}{N_2 T_{INS} + T_{chck}} \right\rfloor$$

(Integral) Number of instructions executed concurrently during 33rd instruction execution in polling is given by:

$$= \left\lfloor \frac{N_1 T_{INS}}{N_2 T_{INS} + T_{chck}} \right\rfloor * N_2 = (K_{poll} N_2)$$

Hence total time

$$T_{Poll}^{net} = 32 * 6T_{clk} + T_{33} + (67 - K_{Poll} N_2)\, T_{INS}$$

$$\begin{aligned} T_{Poll}^{net} &= 192T_{clk} + (63 + 6N_1)\, T_{clk} + (67 - K_{Poll} N_2)\, 6T_{clk} \\ T_{Poll}^{net} &= (192 + 63 + 402 + 6N_1 - 6K_{poll} N_2)\, T_{clk} \\ T_{Poll}^{net} &= (657 + 6N_1 - 6K_{poll} N_2)\, T_{clk} \end{aligned} \tag{2}$$

$$\begin{aligned} T_{Poll}^{net} &= \left(657 + 6N_1 - 6\frac{N_1 T_{INS}}{N_2 T_{INS} + T_{chck}} N_2\right) T_{clk} \\ T_{Poll}^{net} &= \left(657 + 6N_1 - 6\frac{N_1 3}{N_2 3 + 1} N_2\right) T_{clk} \\ T_{Poll}^{net} &= \left(657 + \frac{6N_1}{3N_2 + 1}\right) T_{clk} \end{aligned}$$

give full mark

Case C: Interrupt

$$\begin{aligned} T_{Intrrpt}^{net} &= 32 * 6T_{clk} + T_{33} + (67 - N_1)\, T_{INS} \\ T_{Intrrpt}^{net} &= 192T_{clk} + (63 + 6N_1)\, T_{clk} + (67 - N_1)\, 6T_{clk} \\ T_{Intrrpt}^{net} &= (192 + 63 + 402 + 6N_1 - 6N_1)\, T_{clk} \\ T_{Intrrpt}^{net} &= (192 + 63 + 402)\, T_{clk} = 657T_{clk} \\ T_{Intrrpt}^{net} &= 657T_{clk} \end{aligned} \tag{3}$$

Just mentioning inequality 1 mark. Remaining 2 marks for establishing.

(b) We show that $T^{net}_{prptl} > T^{net}_{poll} > T^{net}_{intrrpt}$. The first inequality follows from equ 1 & 2. Alternatively, $T^{net}_{prptl} = (657 + 6N_1)\,T_{clk} > \left(657 + \frac{6N_1}{3N_2+1}\right) T_{clk} = T^{net}_{Poll}$. The second inequality follows from equs 2 & 3, by noting that the quantity $6N_1 - 6K_{poll}N_2$ is positive, since $N_1 > K_{poll}N_2$ (follows from the very definition of K_{poll}). $(657 + 6N_1)\,T_{clk} > (657 + 6N_1 - 6K_{poll}N_2)\,T_{clk} > 657T_{clk}$. Or $6N_1 > 6N_1 - 6K_{poll}N_2T_{clk} > 0$. Therefore, the most efficient scheme is the interrupt as more number of instructions are parallelly executed by the CPU during the I/O operation at 33rd instruction execution interval. (3 marks)

(c) For case A, all instructions greater than 33 are executed after 33rd instruction (because of serial nature). Hence, CPU would not wait.

For case B of polling, CPU would not wait to execute K^{th} instruction, only if the time taken for instructions after 33 till K is greater than $T_{I/O}$ which is given under

$$(K - 33)\,6T_{Clk} > K_{poll}N_2 6T_{Clk}$$

or

$$(K - 33) > \left\lfloor \frac{N_1 T_{INS}}{N_2 T_{INS} + T_{chck}} \right\rfloor N_2$$

$$(K - 33) > \frac{3N_1}{3N_2 + 1} N_2$$

The N_1 and N_2 should satisfy the above equations so that the CPU need not wait for results of 33rd instruction so that Kth instruction can be executed as and when its turn appears.

For case C of interrupt, similar expression is

$$(K - 33)\,6T_{Clk} > N_1\,(6T_{Clk})$$

and hence N_1 has to satisfy,

$$(K - 33) > N_1$$